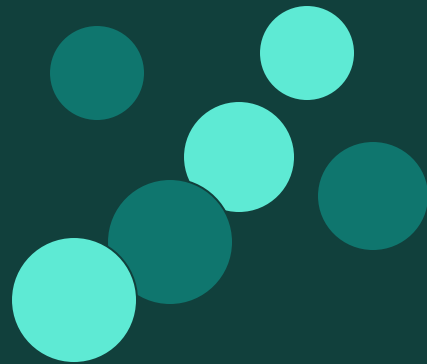


PYTHON을 이용한 데이터 분석 · 13강

앙상블 학습

Random Forest & Gradient Boosting

이론편 — 나무 한 그루보다 숲이 강하다



오늘의 학습 목표



앙상블의 개념 이해

여러 개의 약한 모델을 합쳐 하나의 강한 모델을 만드는 원리



Random Forest (배깅)

여러 트리를 독립적으로 만들고 투표로 결정하는 방법



Gradient Boosting (부스팅)

이전 모델의 실수를 순차적으로 보완해가는 방법



특성 중요도와 튜닝

어떤 특성이 중요한지 해석하고 하이퍼파라미터 조정하기

복습 — 나무 한 그루의 한계

10강에서 배운 **의사결정트리**는 이해하기 쉽고 강력했지만, 치명적인 약점이 있습니다.



불안정하다

학습 데이터가 조금만 바뀌어도 트리 모양이 완전히 달라짐



과적합되기 쉽다

너무 깊게 자라면 학습 데이터를 '외워버림'



작은 실수가 크게 번짐

상단에서 한 번 잘못 나뉘면 하단 전체가 영향을 받음

그래서 "나무 한 그루"에 의존하지 말고, "나무 여러 그루(숲)"의 의견을 모으면 어떨까? — 이것이 오늘의 아이디어입니다.

앙상블이란? — 집단지성의 힘

정의 여러 개의 모델(약한 학습기)을 **결합**해서 하나의 모델보다 **더 강력하고 안정적인 예측**을 만드는 기법입니다.

비유: 퀴즈쇼 "백만장자"의 청중에게 물어보기

전문가 한 명보다, 방청객 100명의 다수결이 정답을 맞힐 확률이 더 높았다는 유명한 통계가 있습니다. 각자는 불완전해도, 여럿이 모이면 오류가 상쇄됩니다.



Tree 1
예측: A



Tree 2
예측: A



Tree 3
예측: B

→ 다수결

최종 예측: A (2표 vs 1표)

앙상블의 두 전략 — Bagging vs Boosting

Bagging (배깅)

방식 여러 트리를 병렬로, 독립적으로 학습

데이터 무작위 복원추출(부트스트랩)로 각기 다른 샘플 사용

결합 다수결(분류) / 평균(회귀)

대표 알고리즘 Random Forest

강점 과적합을 줄이고 안정성을 높임

Boosting (부스팅)

방식 여러 트리를 순차적으로, 이전 실수를 보완하며 학습

데이터 이전 모델이 틀린 부분에 가중치를 더함

결합 가중합 (성능 좋은 모델에 더 큰 비중)

대표 알고리즘 Gradient Boosting

강점 편향(bias)을 줄이고 정확도를 높임

Random Forest — 나무들의 숲



여러 개의 의사결정트리를 만들고, 그 트리들의 다수결로 최종 예측을 결정하는 앙상블 기법입니다.

이름에 "Random"이 들어간 이유 — 두 가지 무작위성

① 데이터 무작위 선택

각 트리마다 전체 데이터에서 복원추출로 다른 샘플을 뽑아 학습 (부트스트랩)

② 특성 무작위 선택

각 분기마다 전체 특성 중 일부만 무작위로 고려 (트리들을 서로 다르게 만듦)

→ 서로 다른 데이터 + 서로 다른 특성으로 자란 트리들은 서로 다른 실수를 하고, 그 실수들이 다수결에서 상쇄됩니다.

Random Forest 작동 흐름

1

부트스트랩 샘플링

2

트리 N개 학습

3

각 트리가 예측

4

다수결/평균

원본 데이터에서 무작위 복원추출로 N개의 샘플셋을 생성
각 샘플셋마다 독립적인 트리를 특성 데이터에 대해 트리 N개가 각각 예측하는 다수결, 회귀는 평균으로 최종 결과 결정
부만으로 학습 수행

핵심 비유

100명의 의사에게 같은 환자를 보여주되, 각 의사에게 조금씩 다른 검사 기록(부트스트랩)과 조금씩 다른 전문 분야(특성 일부)만 보여줍니다. 100명의 진단을 모아 다수결로 최종 진단을 내리면, 한 명의 의사보다 훨씬 안정적입니다.

Random Forest의 강점과 한계



장점 ① 과적합에 강하다

트리 하나보다 훨씬 안정적. 나무가 많을수록 노이즈에 덜 민감.



장점 ② 특성 중요도 제공

어떤 특성이 예측에 중요했는지 자동으로 계산해줌.



장점 ③ 튜닝이 비교적 쉽다

트리 개수를 늘려도 성능이 잘 나빠지지 않음.



한계 ① 해석이 어렵다

트리 하나는 그림으로 그릴 수 있지만, 100개는 불가능 (블랙박스화).



한계 ② 느릴 수 있다

트리가 많을수록 학습·예측 시간이 늘어남.

Gradient Boosting — 실수를 계속 보완하기



트리를 하나씩 순차적으로 추가하면서, 이전 트리들이 놓친 오차(잔차)를 다음 트리가 학습하는 방식입니다.

Random Forest와의 결정적 차이

Random Forest

트리들이 서로 독립적 (병렬). 한 트리가 다른 트리를 몰라도 됨.

Gradient Boosting

트리들이 순서대로 연결됨 (순차). 다음 트리는 반드시 이전 트리들의 오차를 봐야 함.

→ 마치 시험을 채점한 뒤 틀린 문제만 집중적으로 다시 공부하는 것과 같습니다.

Gradient Boosting 작동 흐름

- 1) 첫 번째 얇은 트리를 학습 (대체로 매우 단순, 성능도 낮음)
- 2) 실제값과 예측값의 차이(잔차/오차)를 계산
- 3) 그 오차를 예측하는 새로운 트리를 추가로 학습
- 4) 기존 예측 + 새 트리의 예측(learning_rate만큼 반영)을 더함
- 5) 2~4단계를 정해진 횟수(n_estimators)만큼 반복

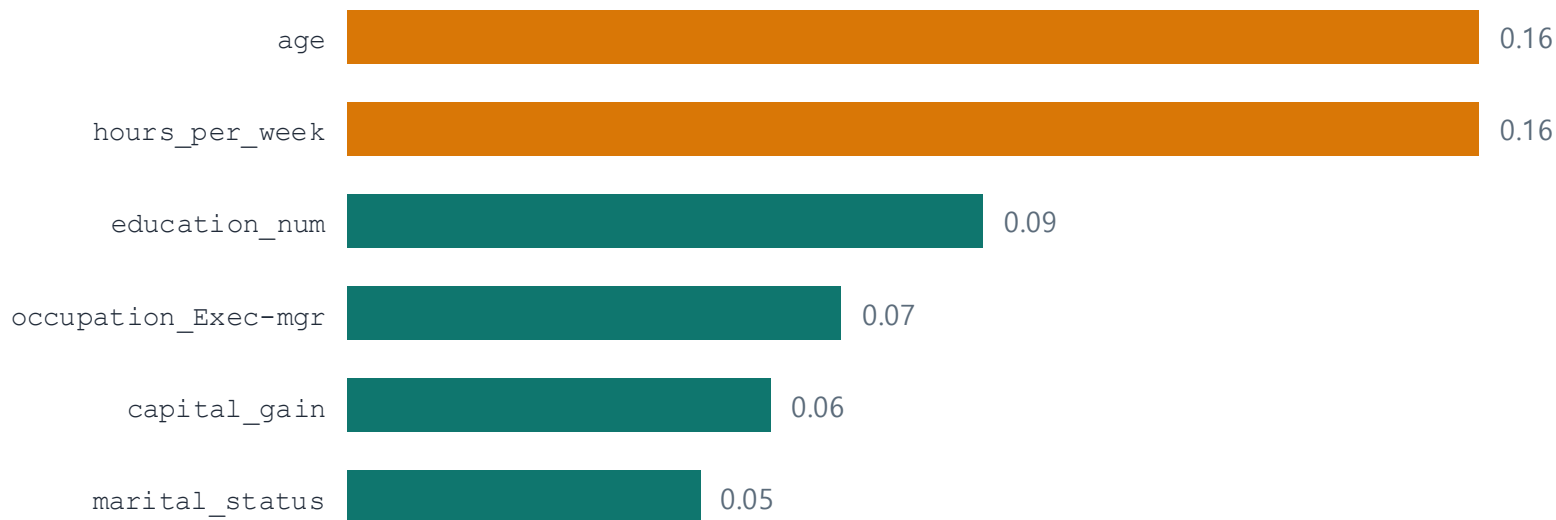
learning_rate 각 트리의 기여도를 얼마나 반영할지 조절하는 값 (보통 0.01~0.3). 작을수록 신중하게, 많은 트리로 천천히 학습합니다.

Random Forest vs Gradient Boosting 종합 비교

	Random Forest	Gradient Boosting
학습 방식	병렬 (독립적)	순차 (이전 오차 보완)
속도	상대적으로 빠름 (병렬화 가능)	상대적으로 느림 (순차적)
과적합 위험	낮음 (트리를 늘려도 안전)	높음 (너무 많으면 과적합)
튜닝 난이도	쉬움 (<code>n_estimators</code> 위주)	까다로움 (<code>learning_rate</code> 등 함께 조정)
대표 성능	안정적, 무난하게 좋음	잘 튜닝하면 최고 성능

특성 중요도 — 앙상블의 강력한 해석 도구

핵심 Random Forest와 Gradient Boosting은 각 특성이 예측에 얼마나 기여했는지를 숫자로 알려줍니다.



`model.feature_importances_` 로 확인 — 데이터를 더 잘 이해하고 불필요한 특성을 제거하는 데 활용

주요 하이퍼파라미터

n_estimators

트리(나무)의 개수

많을수록 안정적이지만 느려짐. 보통 100~500

max_depth

각 트리의 최대 깊이

너무 깊으면 과적합 (10강 복습!)

learning_rate

Boosting 전용 — 학습 보폭

작을수록 신중, n_estimators와 함께 조정 (0.01~0.3)

random_state

무작위성 고정

결과 재현을 위해 항상 고정 (예: 42)

파이썬에서는 어떻게 할까? — 다음 실습 예고

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

rf = RandomForestClassifier(
    n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

gb = GradientBoostingClassifier(
    n_estimators=200, learning_rate=0.1,
    random_state=42)
gb.fit(X_train, y_train)

print(rf.feature_importances_)
```

RandomForestClassifier

n_estimators개의 트리를 병렬로 학습

GradientBoostingClassifier

n_estimators개의 트리를 순차로 학습

fit() / predict()

9~12강과 완전히 동일한 패턴!

패턴 인식 모델 이름만 바뀌었을 뿐, import → 모델 생성 → fit → predict 구조는 9강부터 지금까지 항상 같습니다.

SUMMARY

오늘의 핵심 정리

- ✓ **앙상블 학습** 여러 모델을 결합해 하나의 모델보다 강력한 예측을 만드는 기법
- ✓ **Bagging (Random Forest)** 여러 트리를 병렬·독립적으로 학습 후 다수결/평균
- ✓ **Boosting (Gradient Boosting)** 여러 트리를 순차적으로, 이전 오차를 보완하며 학습
- ✓ **특성 중요도** 어떤 특성이 예측에 중요했는지 자동으로 계산해줌
- ✓ **핵심 하이퍼파라미터** `n_estimators`, `max_depth`, `learning_rate`로 성능 조절

NEXT 13강 실습 — 성인 소득 예측(Adult Income)과 Titanic으로 RF·GB 직접 구현

PYTHON을 이용한 데이터 분석 · 13강 실습

파이썬으로

Random Forest·Gradient Boosting 구현

```
rf = RandomForestClassifier()  
gb =  
GradientBoostingClassifier()  
  
Accuracy: RF 0.77 / GB 0.69
```

성인 소득 예측(*Adult Income*)과 *Titanic*으로 앙상블 모델을 깊이 있게 탐구

환경 설정 체크리스트

1 Import 확인

```
pandas / sklearn.ensemble / sklearn.tree / sklearn.model_selection / metrics
```

2 Adult Income 로드

```
income_df = pd.read_csv('adult_income.csv')
```

3 Titanic 로드

```
titanic = pd.read_csv('titanic.csv')
```

4 구조 확인

```
income_df.head() / income_df.info() / income_df['income'].value_counts()
```

5 10강 트리 복습

```
DecisionTreeClassifier 사용법 기억나는지 확인
```

라이브러리 импорт 및 데이터 로드

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score

income_df = pd.read_csv('adult_income.csv')
print(f'전체 데이터: {len(income_df)}명')
print(income_df['income'].value_counts())
```

성인 소득 예측 데이터 탐색



1,000명의 인구조사 데이터로 '연소득이 5만 달러를 넘는(>50K)'을 예측합니다.

주요 컬럼

<code>age</code>	나이
<code>education_num</code>	교육 연수 (숫자)
<code>occupation</code>	직업 (범주형)
<code>hours_per_week</code>	주당 근무시간
<code>capital_gain</code>	자본 이득
<code>income</code>	타겟: $\leq 50K$ / $> 50K$

탐색 순서

1. `df.head()` / `df.info()`
2. `income` 분포 확인 (불균형?)
3. 범주형 컬럼 목록 확인
4. `education_num` vs `income` 관계
5. 결측치 확인

탐색 코드 — 범주형 컬럼과 소득 분포

```
print(income_df.info())
print(income_df.describe())

# 범주형 컬럼 확인
cat_cols = income_df.select_dtypes(include='object').columns
print(f'범주형 컬럼: {list(cat_cols)}')

# 소득 분포 (불균형 데이터인지 확인) — 약 38.7%만 고소득
print(income_df['income'].value_counts(normalize=True))

# 교육 연수별 평균 소득 비율 (9th 19% → Doctorate 74%)
print(income_df.groupby('education')['income'].apply(
    lambda x: (x=='>50K').mean()))
```

전처리 — 범주형 데이터 인코딩

⚠ **workclass, education, marital_status, occupation**은 모두 문자! 숫자로 바꿔야 합니다.

원-핫 인코딩 (One-Hot Encoding)

범주 하나마다 0/1 컬럼을 새로 만드는 방법. pandas의 `get_dummies()` 함수로 간단히 처리 가능합니다.

예: `occupation_Sales, occupation_Tech-support ...` 컬럼이 자동 생성됨

```
X = pd.get_dummies(
    income_df.drop(
        ['id', 'income'], axis=1),
    drop_first=True)

y = (income_df['income']
     == '>50K').astype(int)

print(X.shape)  # (1000, 26)
```

특성/타겟 분리 및 Train-Test Split

```
# 특성(X)과 타겟(y) 만들기
X = pd.get_dummies(income_df.drop(['id', 'income'], axis=1), drop_first=True)
y = (income_df['income'] == '>50K').astype(int) # >50K=1, <=50K=0

print(f'특성 개수: {X.shape[1]}개') # 원래 9개 → 인코딩 후 26개

# Train-Test Split (9~12강과 동일한 패턴)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

print(f'학습: {len(X_train)}명 / 테스트: {len(X_test)}명') # 700 / 300
```

기준점(Baseline) 만들기 — 트리 1개

앙상블이 얼마나 더 좋은지 비교하려면, 먼저 트리 1개의 성능을 알아야 합니다.

```
# 10강에서 배운 단일 의사결정트리
tree = DecisionTreeClassifier(random_state=42, max_depth=6)
tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)
acc_tree = accuracy_score(y_test, y_pred_tree)

print(f'단일 트리 정확도: {acc_tree:.2%}') # 약 73.00% - 기준점!
```

이 정확도를 기억해두세요. 잠시 후 Random Forest, Gradient Boosting과 비교합니다!

나무 한 그루, 직접 그려보기

숲을 만들기 전에, 그 재료가 되는 나무 한 그루가 실제로 어떻게 생겼는지 시각화합니다.

```
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))
plot_tree(tree, max_depth=3, # 상위 3단계만 표시 (전체는 너무 복잡함)
          feature_names=X.columns,
          class_names=['<=50K', '>50K'],
          filled=True, fontsize=8)
plt.title('단일 Decision Tree (상위 3단계)')
plt.show()
```

생각해보기 이 나무 한 그루는 그림으로 그릴 수 있어 이해하기 쉽지만, Random Forest는 이런 나무를 200그루나 만듭니다. 200장을 다 볼 수 있을까요?

Random Forest 학습하기



나무 200그루를 병렬로 심고, 다수결로 결정합니다.

```
rf = RandomForestClassifier(  
    n_estimators=200,    # 트리 200그루  
    max_depth=10,  
    random_state=42)  
rf.fit(X_train, y_train)  
  
y_pred_rf = rf.predict(X_test)  
acc_rf = accuracy_score(y_test, y_pred_rf)  
print(f'Random Forest 정확도: {acc_rf:.2%}') # 약 76.67%
```

트리 개수(n_estimators)를 바꾸면?

```
accs = []
tree_counts = [10, 50, 100,
                200, 400]

for n in tree_counts:
    m = RandomForestClassifier(
        n_estimators=n,
        random_state=42)
    m.fit(X_train, y_train)
    acc = accuracy_score(
        y_test, m.predict(X_test))
    accs.append(acc)
    print(f'n={n}: {acc:.2%}')
```

관찰 포인트

트리가 10개일 때보다 50~100개일 때가 보통 더 안정적입니다.

그런데 400개와 100개의 차이는 크지 않을 수 있습니다
— 어느 순간부터는 '수확체감'!

💡 Random Forest는 트리를 늘려도 과적합 위험이 크게 늘지 않는다는 것도 확인해보세요.

OOB Score — 숨겨진 '공짜' 검증 데이터

각 트리는 부트스트랩으로 전체 데이터의 약 63%만 사용합니다. 나머지 37%는 어떻게 될까요?

```
rf_oob = RandomForestClassifier(  
    n_estimators=200,  
    max_depth=10,  
    random_state=42,  
    oob_score=True) # 핵심!  
rf_oob.fit(X_train, y_train)  
  
print(f'OOB 점수: {rf_oob.oob_score_:.2%}')  
# 약 70.71% - 테스트 세트 없이 얻은 추정치!
```

OOB(Out-of-Bag)란?

각 데이터 포인트는 전체 트리 중 일부(약 37%)에 의해 '한 번도 학습에 쓰이지 않은 채' 남습니다.

그 데이터로 검증하면, 별도의 테스트 세트 없이도 성능을 가늠할 수 있습니다 — Random Forest만의 특별한 기능!

Gradient Boosting 학습하기



얇은 나무를 순서대로 심으며, 이전 나무의 실수를 계속 보완합니다.

```
gb = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.1,      # 학습 보폭  
    max_depth=3,           # 얇은 트리!  
    random_state=42)  
gb.fit(X_train, y_train)  
  
y_pred_gb = gb.predict(X_test)  
acc_gb = accuracy_score(y_test, y_pred_gb)  
print(f'Gradient Boosting 정확도: {acc_gb:.2%}')  # 약 69.00%
```

학습 보폭(learning_rate)을 바꾸면?

```
rates = [0.01, 0.05, 0.1,
         0.3, 0.5]

for lr in rates:
    m = GradientBoostingClassifier(
        n_estimators=200,
        learning_rate=lr,
        random_state=42)
    m.fit(X_train, y_train)
    acc = accuracy_score(
        y_test, m.predict(X_test))
    print(f'lr={lr}: {acc:.2%}')
```

관찰 포인트

너무 작은 lr(0.01): 학습이 느려서 200개 트리로는 부족할 수 있음

적당한 lr(0.05): 이 데이터에서는 오히려 0.1보다 나은 결과(약 74%)가 나올 수 있음

너무 큰 lr(0.5): 성급하게 학습해서 성능이 흔들릴 수 있음

💡 기본값(0.1)이 항상 최선은 아닙니다 — 튜닝이 필요한 이유!

미니 그리드서치 — 두 하이퍼파라미터 동시 탐색

`max_depth`와 `n_estimators`를 동시에 바꿔가며 최적 조합을 찾아봅니다.

```
best_acc, best_combo = 0, None
for md in [2, 3, 4]:
    for ne in [100, 200, 300]:
        m = GradientBoostingClassifier(n_estimators=ne, max_depth=md,
                                       learning_rate=0.05, random_state=42)
        acc = accuracy_score(y_test, m.fit(X_train, y_train).predict(X_test))
        if acc > best_acc: best_acc, best_combo = acc, (md, ne)
print(f'최고 조합: max_depth={best_combo[0]}, n_estimators={best_combo[1]} → {best_acc:.2%}')
```

실제 결과표 (learning_rate=0.05 고정)

max_depth=2 → [0.740, 0.747, 0.737] max_depth=3 → [0.733, 0.740, 0.710] max_depth=4 → [0.733, 0.683, 0.677]

→ 최고 조합: max_depth=2, n_estimators=200 (74.67%). 알을수록 오히려 안정적이었습니다!

3개 모델 성능 한눈에 비교하기

```
models = ['Decision Tree', 'Random Forest', 'Gradient Boosting']
accuracies = [acc_tree, acc_rf, acc_gb]

plt.figure(figsize=(9, 6))
bars = plt.bar(models, accuracies,
               color=['#94A3B8', '#0F766E', '#D97706'])
plt.ylabel('정확도')
plt.title('성인 소득 예측 - 모델 성능 비교')
plt.ylim([0.6, 1.0])

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_height() / 2, bar.get_height(), f'{acc:.1%}')
plt.show()
```

특성 중요도 — 무엇이 소득을 결정할까?

```
importances = pd.Series(rf.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=False)

print(importances.head(10)) # 상위 10개 특성

importances.head(10).plot(kind='barh', color='teal')
plt.title('Random Forest 특성 중요도 Top 10')
plt.gca().invert_yaxis()
plt.show()
```

실제 결과 age, hours_per_week가 1·2위, education_num이 3위입니다. 연속형 특성이 분기 기회가 많아 상위권에 오르기 쉽다는 점도 기억하세요.

혼동행렬 & ROC-AUC — 정확도 너머 보기

12강에서 배운 혼동행렬을 다시 꺼내, 불균형 데이터(38.7%만 고소득)에 더 적합한 지표까지 확인합니다.

```
print(confusion_matrix(
    y_test, rf.predict(X_test)))
# [[169  17]
#   [ 53  61]]

print(classification_report(
    y_test, rf.predict(X_test)))
# precision 0.78 / recall 0.54
# (>50K 클래스 기준)
```

ROC-AUC란?

모델이 실제 정답과 예측 확률의 순서를 얼마나 잘 맞추는지 0.5(무작위)~1.0(완벽)로 나타낸 지표입니다.

```
roc_auc_score(y_test,
    rf.predict_proba(X_test)[:,1])
```

실제 결과: RF 0.803 / GB 0.759 — 정확도만 볼 때보다 RF의 우위가 더 뚜렷하게 드러납니다.

Titanic에 같은 프로세스 적용하기



12강에서 썼던 Titanic 데이터를 다시 불러와 RF·GB로 생존 예측을 반복합니다.

```
titanic['Sex'] = (titanic['Sex'] == 'male').astype(int)
X_t = titanic[['Pclass', 'Sex', 'Age', 'Fare']].copy()
X_t['Age'] = X_t['Age'].fillna(X_t['Age'].mean())
y_t = titanic['Survived']

X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(
    X_t, y_t, test_size=0.3, random_state=42)

rf_t = RandomForestClassifier(n_estimators=200, random_state=42)
rf_t.fit(X_train_t, y_train_t)
```

Titanic — 12강 결과와 비교하기

```
gb_t = GradientBoostingClassifier(n_estimators=200, random_state=42)
gb_t.fit(X_train_t, y_train_t)

acc_rf_t = accuracy_score(y_test_t, rf_t.predict(X_test_t))
acc_gb_t = accuracy_score(y_test_t, gb_t.predict(X_test_t))
print(f'RF: {acc_rf_t:.2%} / GB: {acc_gb_t:.2%}') # 약 66.29% / 69.66%
```

12강 기억나시나요? 같은 random_state로 재현한 12강 결과: KNN 62.55%, Decision Tree 73.03%, Logistic Regression 74.16%. 오늘의 RF·GB는 이보다 나아졌을까요? 다음 슬라이드에서 5개 모델을 한 번에 비교합니다!

5개 모델 종합 비교 — 항상 앙상블이 이길까?

12강 모델 (회색)

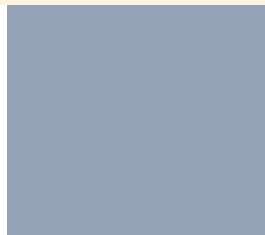
73.0%

74.2%

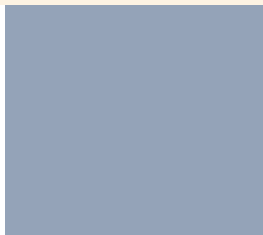
13강 앙상블 (색상)

66.7%

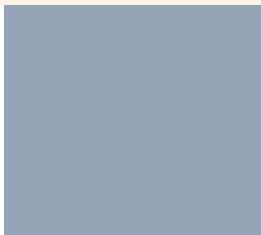
놀라운 발견 Titanic처럼 특성이 4개뿐인 단순한 데이터에서는, 오히려 단일 Decision Tree(73.03%)와 Logistic Regression(74.16%)이 Random Forest(66.29%)·Gradient Boosting(69.66%)보다 더 좋은 성능을 보였습니다. 복잡한 모델이 항상 이기지는 않습니다!



KNN



Tree



Logistic



RF



GB

CHALLENGE 1

과적합 확인 — 학습 정확도 vs 테스트 정확도

TODO: 각 모델에 대해 학습 데이터 정확도와 테스트 데이터 정확도를 모두 계산하고 비교하세요. 차이가 크다면 과적합을 의심할 수 있습니다.

```
for name, m in [('Tree', tree), ('RF', rf), ('GB', gb)]:  
    train_acc = accuracy_score(y_train, m.predict(X_train))  
    test_acc = accuracy_score(y_test, m.predict(X_test))  
    gap = train_acc - test_acc  
    print(f'{name}: train={train_acc:.2%} test={test_acc:.2%} gap={gap:.2%}')
```

gap이 클수록 과적합 위험이 큼 (단, 테스트 정확도가 최종 기준)

CHALLENGE 2

max_depth 제한으로 과적합 줄이기

Gradient Boosting에서 max_depth를 바꿔가며 과적합이 줄어드는지 확인합니다.

```
for depth in [2, 3, 5, 8, None]:  
    m = GradientBoostingClassifier(  
        n_estimators=200,  
        max_depth=depth,  
        random_state=42)  
    m.fit(X_train, y_train)  
    tr = accuracy_score(y_train,  
        m.predict(X_train))  
    te = accuracy_score(y_test,  
        m.predict(X_test))  
    print(depth, tr, te)
```

생각해보기:

max_depth=None → 트리가 끝까지 자람 → 학습 정확도는 100%에 가깝지만 테스트는?

max_depth가 작을수록 → 트리가 얕음(약한 학습기) → Boosting의 원래 취지에 더 가까움

→ Gradient Boosting은 보통 max_depth를 3 안팎의 얇은 값으로 씁니다!

CHALLENGE 3

Titanic 특성 중요도로 스토리 만들기

TODO: Titanic용 Random Forest의 `feature_importances_`를 확인하고, 어떤 특성이 생존에 가장 중요했는지 순위를 매겨보세요. Sex가 상위권일지 예상해보고 실제로 확인하세요.

```
imp_t = pd.Series(rf_t.feature_importances_, index=X_t.columns)
print(imp_t.sort_values(ascending=False))
# 실제로는 Fare·Age가 압도적 1·2위, Sex·Pclass는 하위권일 수 있습니다!
```

CHALLENGE 4

교차검증(Cross-Validation) — 한 번의 분할을 믿을 수 있을까?

`random_state`를 다르게 하면 정확도가 달라질 수 있습니다. 5-Fold 교차검증으로 더 안정적인 평가를 해봅시다.

```
from sklearn.model_selection import
cross_val_score

cv_tree = cross_val_score(
    DecisionTreeClassifier(
        random_state=42, max_depth=6),
    X, y, cv=5)
cv_rf = cross_val_score(rf, X, y, cv=5)
cv_gb = cross_val_score(gb, X, y, cv=5)

print(f'Tree 평균: {cv_tree.mean():.2%}')
print(f'RF 평균: {cv_rf.mean():.2%}')
print(f'GB 평균: {cv_gb.mean():.2%}')
```

5-Fold란?

데이터를 5조각으로 나눠, 매번 다른 조각을 테스트로 쓰고 나머지로 학습 — 5번 반복해 평균을 냅니다.

실제 결과: Tree 70.2%, RF 72.4%, GB 71.6% (평균) — 한 번의 분할(73.0/76.7/69.0%)과는 다른 그림이 보입니다!

CHALLENGE 5

미니 프로젝트 — 나만의 최고 모델 찾기



지금까지 배운 모든 도구를 동원해서, 성인 소득 예측의 최고 성능을 직접 갱신해보세요.

1

RandomForest 또는 GradientBoosting 중 하나를 선택하세요.

2

`n_estimators`, `max_depth`, (GB라면 `learning_rate`)를 자유롭게 조합해보세요.

3

`cross_val_score(cv=5)`의 평균으로 성능을 평가하세요 (한 번의 분할은 믿지 않기!).

4

지금까지 나온 최고 기록(RF 단일분할 76.67% / CV 평균 72.4%)을 넘어보세요.

발표 시간이 된다면 자신이 찾은 최고 조합과 CV 평균 정확도를 옆 사람과 공유해보세요!

실습 완료 확인 사항

- | | | |
|---|-------------------|-------------------------------------|
| 1 | Adult Income 탐색 | 구조, 소득 분포, 범주형 컬럼 확인 |
| 2 | 전처리 | get_dummies로 원-핫 인코딩, X/y 분리 |
| 3 | 기준점 + 트리 시각화 | 단일 Decision Tree 정확도 확보 및 plot_tree |
| 4 | Random Forest | 학습 + n_estimators 실험 + OOB Score |
| 5 | Gradient Boosting | 학습 + learning_rate 실험 + 미니 그리드서치 |
| 6 | 모델 비교 & 심화 평가 | 정확도 비교, 특성 중요도, 혼동행렬·ROC-AUC |
| 7 | Titanic 재적용 | 동일 프로세스 반복 + 12강 3개 모델과 5종 비교 |
| 8 | 도전 문제 | 과적합, max_depth, 특성해석, 교차검증, 미니 프로젝트 |

오늘의 핵심

- Part 0~2** → 데이터 로드, 탐색, 원-핫 인코딩으로 전처리
- Part 3** → 단일 트리로 기준점 확보 + 나무 시각화
- Part 4** → Random Forest 학습, n_estimators 실험, OOB Score
- Part 5** → Gradient Boosting 학습, learning_rate·그리드서치
- Part 6** → 정확도 비교, 특성 중요도, 혼동행렬·ROC-AUC
- Part 7** → Titanic 재적용 — 항상 앙상블이 이기지 않는다!
- 도전 1~5** → 과적합, 튜닝, 해석, 교차검증, 미니 프로젝트